

AI4Land data-loading optimization

What worked, what didn't, and how to store the next dataset

Author: Ferran Mirabent Rubinat **Team:** CES / AI team, BSC **Date:** July 30, 2026

TLDR. Training was data-loader bound: the H100s sat idle >90% of the time, with a ~ 80 s stall every `num_workers` batches. The cost was GPFS *metadata latency*, paid per chunk file opened, independent of its size. The multi-zarr layout opened ~ 93 chunk files per sample. **Two fixes:** (1) stack all variables into one `dynamic` + one `static` array so a sample opens ~ 9 files, not ~ 93 ; (2) read patches through raw `zarr`, bypassing the xarray/dask graph whose per-chunk cost scales with the store's time extent. On the full historical store, 10 epochs on 1 node went from ~ 55 h $\rightarrow \sim 2.3$ h (merge) $\rightarrow \sim 1.3$ h (raw reader). Training is now GPU-bound. All layouts are bit-identical (parity 0), so model accuracy is unchanged.

1. The bottleneck (diagnosis)

Per-sample fetch time factors as $opens/sample \times cost/open$.

- **Opens/sample is set by the layout.** 41 read calls (one per variable per timestep) $\times \sim 2.25$ chunk files touched per random 256^2 read ≈ 93 opens (measured 92.7).
- **Cost/open is GPFS metadata latency, flat in bytes.** Cold open ~ 20 ms, warm ~ 0.1 ms. Per-chunk anatomy: bytes-from-disk ~ 19 – 20 ms, xarray materialise ~ 16 ms, Blosc decode 0.3 ms.
- **Not a bandwidth problem.** Single-stream GPFS ~ 556 MB/s; the ~ 3 MB compressed pulled per sample would take ~ 5 ms if bandwidth-bound, against the ~ 1.25 s observed ($\sim 200\times$ gap).
- **The periodic stall** is structural: the prefetch pool refills all `num_workers` workers at once, so one slow refill lands every `num_workers` batches. More workers hide it (deeper outstanding I/O) but never remove it.
- **Store-size effect.** Bigger store $\rightarrow$ slower per sample, though the slice read is identical. The cause: the xarray/dask graph enumerates the store's *full* time extent, not the slice.

2. What worked

1. **Multi-zarr $\rightarrow$ single store.** One store per refill instead of five. Stalls shrink $\sim 8\times$ (~ 80 s $\rightarrow \sim 10$ s). However, the stalls remain and they increase with store size (the ~ 80 s $\rightarrow \sim 10$ s is between the Multizarr 1900 - 2020 and the single zarr 2000 - 2015).
2. **Single $\rightarrow$ merged (MergedZarrDataset).** Stack the 14 LUH2 + population + Köppen-Geiger into one `dynamic` array (variable as a new axis) and the 6 static into one `static` array. Opens/sample $93 \rightarrow 9$ ($\sim 10\times$), file count $16.9\times$ fewer, $\sim 7\times$ single-thread. Short-window training becomes GPU-bound.
3. **Merged xarray $\rightarrow$ raw-zarr reader (RawMergedZarrDataset, `dataset_type: merged_raw`).** Read patches via `zarr_array.oindex[...]`, no xarray/dask. Removes the per-chunk Python/GIL graph work (the store-size-dependent term): $\sim 7\times$ single-thread over merged-xarray, ~ 1.9 – $2.5\times$ multi-worker (the gain grows with workers), $\sim 0\%$ stalls on training. This is the recommended loader.

Isolated dataloader (single-thread getitem)	Size	Median/sample
Multi-zarr, 5 stores	~680 GB	7970 ms
Single-zarr	66 GB	1329 ms
Single-zarr	435 GB	5513 ms
Merged (dynamic+static stacked)	59 GB	174 ms
Merged	387 GB	286 ms
Merged + raw-zarr reader	387 GB	38 ms

Table 1: Median per-sample fetch, model out of the loop. Merge: $7.6\times$ (small) to $19\times$ (big) over single-zarr. Raw reader: $\sim 7\times$ over merged-xarray. The $66 \rightarrow 435$ GB single-zarr jump ($4.2\times$) is the xarray graph store-size.

End-to-end (full historical store, 10 epochs, 1 node, 12 workers)	Wall time
Five separate stores (original)	~ 55 h
Merged store (xarray)	~ 2.3 h
Merged store + raw-zarr reader	~ 1.3 h

Table 2: All layouts bit-identical (parity 0). Merge $\sim 23\times$; raw reader a further $\sim 1.8\times$ ($\sim 42\times$ total). Final layout is GPU-bound: ~ 51 samples/s/GPU, $\sim 0\%$ stalls, per-GPU throughput flat from 1 to 16 ranks.

3. What was tried and did not work

Tried	Outcome and reason
Disable <code>ProcessSynchronizer</code>	No effect. The zarr v2 read path (<code>_chunk_getitem</code>) never acquires the lock; it guards writes only. Config toggle removed, instance kept defensively.
Offline preprocessing bake-in (fill-NaN, normalize)	~ 0 gain. Preprocessing was $\sim 8\%$ of per-sample cost; the residual is stack/transpose/reshape, which a bake-in cannot remove.
<code>ThreadPoolExecutor</code> over per-variable LUH2 reads	No gain; threads=2 regressed $\sim 23\%$. Signature of Blosc/GIL contention, not I/O that parallelizes this way.
<code>chunks=None</code> (drop <code>dask</code> , keep xarray)	No multi-worker gain. Keeps xarray’s indexing overhead; the win needs xarray removed entirely (raw zarr).
Chunk shape $\neq 512^2$	Worse. 256^2 multiplies chunk count and graph work; $1024^2/2048^2$ become decode/read-amplified. 512^2 is optimal for 256^2 patches.
NVMe staging (copy store to node-local disk)	Discarded. Production store > 213 GB does not fit the 480 GB local NVMe.
Zarr v3 sharding	Low value here. Shrinks file count but not the logical chunk count the graph enumerates; our Blosc decode is already 0.3 ms/chunk, and GPU-native decode is therefore irrelevant.

4. Full-dataset follow-up (post-thesis)

The thesis established the fix on short-window stores and projected that merging alone would keep the GPUs fed at full scale. It did not, quite. On the full merged store (all variables, full time range, built by Joan) the ~ 80 s stalls collapsed to 1–2 s spikes but did not vanish: 10 epochs took ~ 2.3 h. The residual is the xarray store-size term (graph cost grows with the store’s time extent, not the slice). Writing the raw-zarr reader removed it and brought the run to ~ 1.3 h, at which point the GPU, not the data path, sets the pace. Parity held throughout (bit-identical, so accuracy is unchanged), and per-GPU throughput is flat from 1 to 16 ranks.

5. Knobs to optimize a new dataset

Goal: minimize *cold chunk-file opens per sample*, keep chunk files reasonably large, and keep the store’s file count from outgrowing the metadata cache.

Knob	Target
Sanity target	loader delivery $\geq$ GPU consume (~ 53 samples/s/GPU)
Verify	parity (bit-identical) before any speed claim
File opens per sample	single digits ($\leq \sim 10$)
Chunk file size (compressed)	$\geq \sim 1$ MB; avoid < 100 KB
Spatial chunk side C	$\geq 2 \times$ patch side P ; expected chunks/read = $(1 + P/C)^2$
Store count	one ARCO store, not per-modality
Variable layout	dynamic stacked on a variable axis; static stacked separately
Reader	raw zarr (<code>oindex</code>), not xarray
Codec	Blosc(lz4) fine; decode negligible

Table 3: Reference card. When in doubt, cut opens/sample first, it pays off in every cache state.